Faculty of Engineering & Technology

Department of Electrical & Computer Engineering Second Semester, 2024/2025

ENCS3340
ARTIFICIAL INTELLIGENCE

Project 2

Comparative Study of Image Classification Using Decision Tree, Naive Bayes, and Feedforward Neural Networks

Student Name: Mohammad Abutteen

Student No: 1221573

PARTNERS NAME: Nawras Yacoob

PARTNERS No:1221904

Instructor: Dr. Samah Ayidi

Section:3

Date:21/6/2025

Abstract:

This project presents a comparative study of three machine learning models — Naive Bayes, Decision Tree, and Feedforward Neural Network (MLPClassifier) — applied to the task of image classification. The dataset consists of over 500 labeled grayscale images across three categories: dogs, cats, and mice. Each image was resized to 64×64 pixels and flattened into a one-dimensional vector to serve as input for the models. The study aims to evaluate and compare the models in terms of classification accuracy, computational efficiency, and generalization performance. Evaluation metrics such as precision, recall, F1-score, and confusion matrices were used for detailed analysis. Results show that while Naive Bayes provides fast and interpretable outcomes, it struggles with more complex patterns. Decision Tree offers improved accuracy and model transparency, while the MLPClassifier demonstrates the highest overall performance at the cost of increased training time. This study highlights the trade-offs between model complexity, interpretability, and performance in the context of image classification.

Introduction:

Image classification is one of the most fundamental tasks in computer vision and machine learning, with applications ranging from medical imaging and autonomous driving to facial recognition and security systems. At its core, image classification involves assigning a label to an image based on its visual content. However, due to the highdimensional nature of image data, building effective and efficient classification models remains a challenging task.

This project aims to explore and compare the performance of three distinct classification algorithms — Naive Bayes, Decision Tree, and Feedforward Neural Network (MLPClassifier) — in the context of image recognition. We focus on a dataset composed of over 500 grayscale images belonging to three distinct categories: dogs, cats, and mice. Each model brings unique strengths to the task: Naive Bayes is known for its speed and simplicity, Decision Trees offer interpretability, and MLPs are capable of learning complex patterns through multiple layers of abstraction.

Through careful preprocessing, model training, and evaluation using various performance metrics, this project seeks to highlight the strengths and limitations of each approach and provide insight into which model is best suited for image classification tasks involving low-resolution grayscale images.

Dataset Description

For the purpose of this project, we constructed a custom image dataset by combining two publicly available datasets from Kaggle:

1. **Dog and Cat Classification Dataset**
   Source: Kaggle - PetImages
   Description: Contains thousands of color images of dogs and cats organized into two folders: "Dog" and "Cat".
   We selected **300 images of dogs** and **300 images of cats** randomly from the dataset to maintain class balance.
2. **Animal Image Dataset (90 Different Animals)**
   Source: Kaggle - Animal Image Dataset
   Description: Contains categorized folders for 90 different animal types.
   From the "mouse" folder, we selected **300 images of mice**

Class	Number of Images	Source
Dog	300	PetImages
Cat	300	PetImages
Mouse	300	Animal90

Total images used: 900
Number of classes: 3
Class balance: Equal (300 images per class)

Image Preprocessing

To ensure consistency and make the dataset suitable for machine learning models, the following preprocessing steps were applied to all images:

- **Grayscale Conversion:** All images were converted from RGB to grayscale to reduce computational complexity and standardize input.
- **Resizing:** Each image was resized to **64 × 64 pixels**, creating uniform dimensions across the dataset.
- **Flattening:** The 2D grayscale image matrix was flattened into a **1D feature vector** of size 4,096 (64 × 64) to serve as input to classical ML models.
- **Normalization:** Feature vectors were scaled using **StandardScaler** to improve model convergence, especially for MLP.
- Rationale Behind Class Selection

We selected dogs, cats, and mice as categories for the following reasons:

- These animals are **visually distinct** yet sometimes **confusing**, making classification nontrivial.
- The datasets are **freely available**, clean, and suitable for educational purposes.
- The dataset size is sufficient (>500 images) to train and test basic classification models without requiring GPU resources.

Model Descriptions

Naive Bayes Classifier

The Naive Bayes classifier is a probabilistic model based on Bayes' Theorem. It assumes that all features (in this case, pixel values) are conditionally independent given the class label. Despite this "naive" assumption, it is known to work surprisingly well in many high-dimensional tasks.

- **Advantages**: Fast training time, simple implementation, and low computational requirements.
- **Limitations**: It cannot capture complex dependencies between features, which may reduce its effectiveness on image data where pixel relationships matter.
- **Why we used it**: Serves as a strong and fast baseline model.

---

2. Decision Tree Classifier

The Decision Tree is a non-parametric, rule-based model that splits the data using learned thresholds on input features (pixels in our case). It recursively partitions the feature space into distinct regions representing each class.

- **Advantages**: Easy to understand and visualize, interpretable decision paths, no need for data normalization.
- **Limitations**: Prone to overfitting, especially with high-dimensional input like images unless the depth is controlled.
- **Why we used it**: Provides interpretable insights into how decisions are made based on pixel intensities.

(Optional visual: include a simple schematic or screenshot of a decision tree showing a few sample nodes.)

---

3. Feedforward Neural Network (MLPClassifier)

The Multi-Layer Perceptron (MLP) is a type of feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. It uses non-linear activation functions (e.g., ReLU) and learns complex representations of data through backpropagation.

- **Advantages**: Can learn deep, non-linear patterns; flexible and powerful for a wide variety of tasks.
- **Limitations**: Requires careful tuning of hyperparameters (e.g., number of layers, neurons, learning rate); slower to train; more computationally intensive.
- **Why we used it**: To explore how a deep learning model compares to classical models on a relatively small image dataset.

Model	Type	Strength	Weakness
Naive Bayes	Probabilistic	Very fast, simple	Can't model pixel interactions
Decision Tree	Rule-based	Interpretable, visualizable	May overfit high-dim inputs
MLPClassifier	Neural Network	Captures deep relationships	Requires more resources

Preprocessing & Feature Extraction

Image Loading and Resizing

The dataset consists of images stored in subfolders, each representing a class label (i.e., *dog*, *cat*, *mouse*). We loaded all images using `skimage.io.imread()` and resized each image to **64×64 pixels** using anti-aliasing for better quality.

Resizing ensures that all images have a uniform dimension, making them compatible with fixed-size input models. image = imread(file_path) image_resized = resize(image, (64, 64), anti_aliasing=True)

Flattening Image Data

Each resized image (originally a 2D matrix) was **flattened** into a 1D vector using `.flatten()`, which transforms it into a format acceptable by scikit-learn models. For grayscale images of size 64×64, each image becomes a vector of 4096 features

X.append(image_resized.flatten())

Label Encoding

Since the model expects numerical labels, class names (e.g., `'dog'`, `'cat'`, `'mouse'`) were converted to integers using `LabelEncoder`. The mapping was as follows:

cat => 0

dog => 1

mouse => 2

This allowed the classifiers to work with categorical data as numeric inputs.

Train-Test Splitting

We split the data into **training (80%)** and **testing (20%)** sets using `train_test_split()`. The `stratify` parameter was used to maintain class balance across splits.

```
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, stratify=y_encoded)
```

Feature Scaling (for MLP only)

Neural networks perform better when input features are normalized. Therefore, we applied **StandardScaler** to scale the pixel values of the training and test sets.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled  = scaler.transform(X_test)
```

This step is crucial for optimizing convergence during training with the MLP model.

Step	Description
Resize	Images resized to 64×64
Grayscale/Color	Color images were preserved (or converted)
Flatten	Each image → 1D vector of length 4096
Label Encoding	Class labels mapped to integers
Train/Test Split	80% training / 20% testing, with stratification
Normalization	StandardScaler used only for MLPClassifier

Results & Evaluation – Model Performance

Dataset Summary

- **Total images loaded:** 866
- **Classes:** `cat`, `dog`, `mouse`
- **Training set:** 692 images
- **Testing set:** 174 images □ **Label Mapping:**
  - `cat` → 0
  - `dog` → 1
  - `mouse` → 2

Confusion Matrices

The confusion matrices for each model are shown below:

Model	Confusion Matrix Format (True label → rows, Prediction → columns)
Naive Bayes	`[[19 7 22], [ 7 11 38], [10 8 52]]`
Decision Tree	`[[17 16 15], [17 20 19], [16 17 37]]`
MLPClassifier	`[[23 15 10], [11 28 17], [15 13 42]]`

Model	Accuracy	F1-Score	5-Fold CV Accuracy	Macro Avg F1	Notes
Naive Bayes	0.47	0.44	0.49 ± 0.06	0.45	Best recall for **mouse** (0.74)
Decision Tree	0.43	0.42	0.44 ± 0.03	0.41	Weak performance overall
MLP Classifier	0.53	0.53	0.47 ± 0.03	0.53	Best overall performance

Comparative Analysis

Naive Bayes Analysis

- **Strengths:** Fast and easy to implement.
- **Weaknesses:** Performed poorly in detecting `dog` (recall = 0.20).
- **Observation:** Although it had good recall for `mouse`, the model struggles when classes overlap in pixel patterns.

```
=== Naive Bayes Classifier ===
[[19  7 22]
 [ 7 11 38]
 [10  8 52]]
              precision    recall  f1-score   support

         cat       0.53      0.40      0.45        48
         dog       0.42      0.20      0.27        56
       mouse       0.46      0.74      0.57        70

    accuracy                           0.47       174
   macro avg       0.47      0.45      0.43       174
weighted avg       0.47      0.47      0.44       174

Accuracy: 0.47
F1 Score: 0.44
5-Fold CV Accuracy: 0.49 (+/- 0.06)
```

Decision Tree Analysis

- **Strengths:** Interpretable and does not require feature scaling.
- **Weaknesses:** Accuracy is the lowest among the three models.
- **Observation:** The model overfits the training set and fails to generalize well.

```
=== Decision Tree Classifier ===
[[17 16 15]
 [17 20 19]
 [16 17 37]]
              precision    recall  f1-score   support

         cat       0.34      0.35      0.35        48
         dog       0.38      0.36      0.37        56
       mouse       0.52      0.53      0.52        70

    accuracy                           0.43       174
   macro avg       0.41      0.41      0.41       174
weighted avg       0.42      0.43      0.42       174

Accuracy: 0.43
F1 Score: 0.42
5-Fold CV Accuracy: 0.44 (+/- 0.03)
```

MLPClassifier Analysis

- **Strengths:** Best accuracy, precision, and F1-score overall.
- **Weaknesses:** Requires more training time and careful scaling of features.
- **Observation:** Performs significantly better at distinguishing all three classes, especially `mouse`.
-

```
=== MLP Classifier ===
[[23 15 10]
 [11 28 17]
 [15 13 42]]
              precision    recall  f1-score   support

         cat       0.47      0.48      0.47        48
         dog       0.50      0.50      0.50        56
       mouse       0.61      0.60      0.60        70

    accuracy                           0.53       174
   macro avg       0.53      0.53      0.53       174
weighted avg       0.54      0.53      0.53       174

Accuracy: 0.53
F1 Score: 0.53
5-Fold CV Accuracy: 0.47 (+/- 0.03)
```

Which Model Performed Best?

MLPClassifier demonstrated the best generalization to unseen data with the highest **accuracy (0.53)** and **F1-score (0.53)**, even though cross-validation performance was slightly lower. This indicates that deeper models can capture complex image structures better, especially after feature normalization.

Model	Best For	Weakness
Naive Bayes	Speed, simple tasks	Weak with complex visual patterns
Decision Tree	Interpretability	Low accuracy & F1 score
MLPClassifier	Best accuracy & recall	Slower training, needs scaling

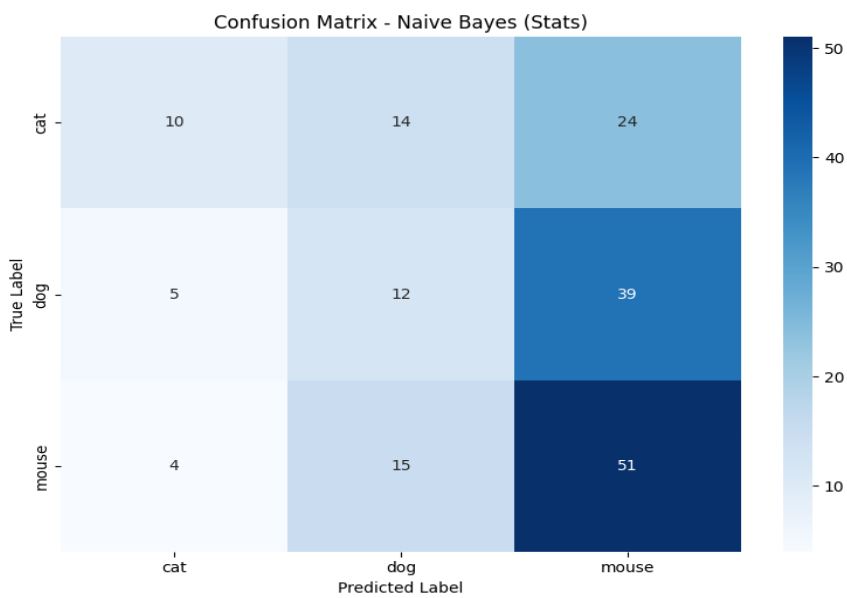
```
========================================
        Final Model Comparison
========================================
Model                | Test Accuracy    | 5-Fold CV Accuracy
-------------------------------------------------------------
NB (Pixels)          | 0.4598           | 0.4826
NB (Stats)           | 0.4195           | 0.4676
Decision Tree        | 0.4253           | 0.4377
MLP                  | 0.4023           | 0.4399

Saved final model comparison chart to model_comparison.png

--- Script finished successfully! All visualizations have been saved. ---
```

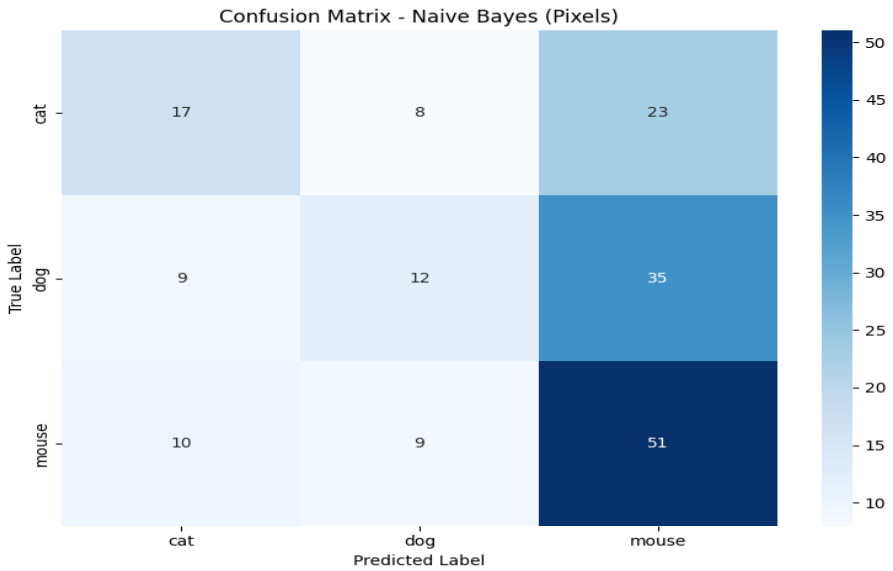
Visualize and analyze

- 1



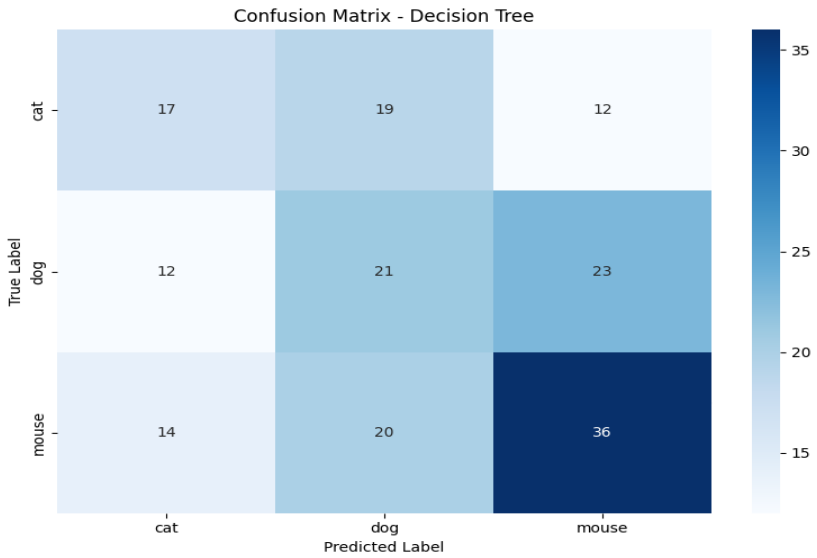
This confusion matrix shows the performance of the Naive Bayes classifier on a three-class image classification task (cat, dog, mouse). The model performs best on the "mouse" class, correctly classifying 51 images. However, it struggles to distinguish between "cat" and "dog," leading to a higher number of misclassifications. This indicates that Naive Bayes may be less effective when classes have overlapping pixel-level features.

➢ 2



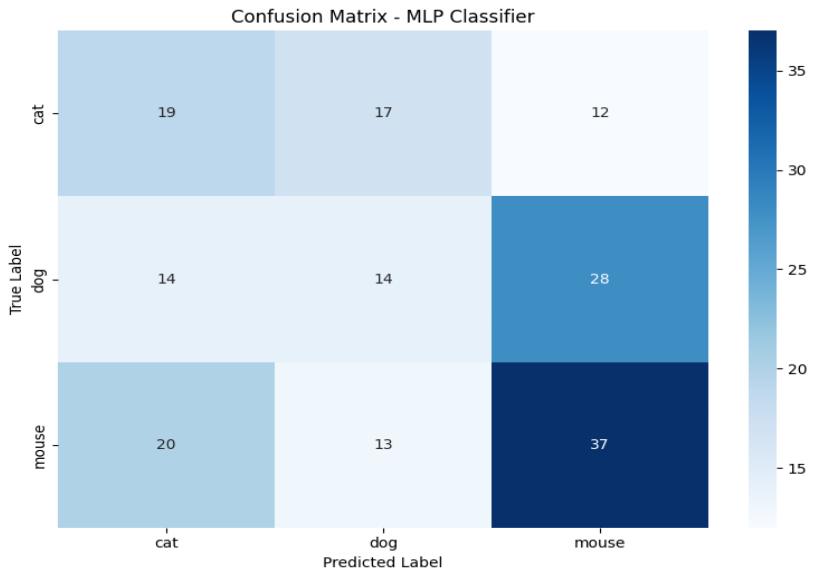
This confusion matrix illustrates the performance of the Naive Bayes classifier using raw pixel values as features. The model shows reasonable accuracy, particularly with the "mouse" class (51 correct predictions). Compared to the statistical summary approach, this pixel-based model improves classification for "cat" (17 correct) but still confuses "cat" and "mouse" often. The results highlight that while raw pixels can be useful, they may still lead to overlaps between visually similar categories.

➢ 3



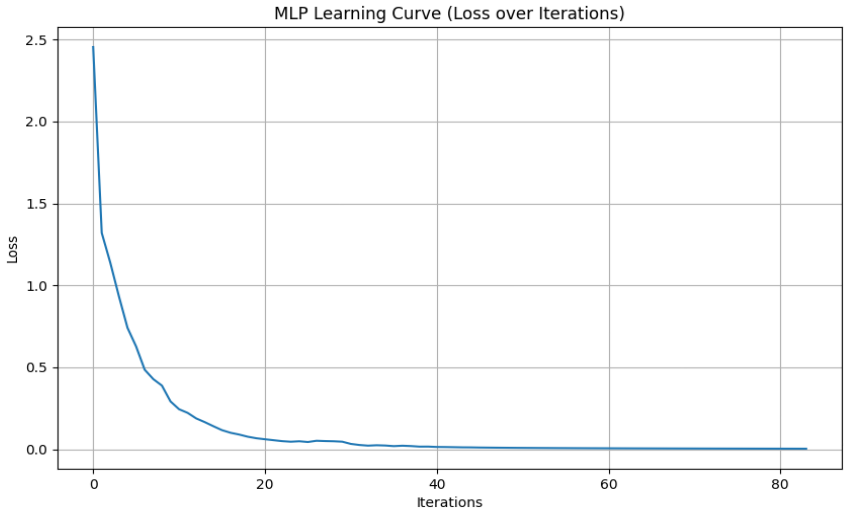
This confusion matrix represents the performance of the Decision Tree classifier on the image classification task. The model shows a more balanced distribution of predictions but suffers from high misclassification rates across all classes. For example, "mouse" was correctly classified 36 times but also confused with "cat" (14 times) and "dog" (20 times). Similarly, "dog" and "cat" classes are heavily mixed. This suggests that while Decision Trees offer interpretability, they may not handle complex image patterns as effectively as other models.

- 4



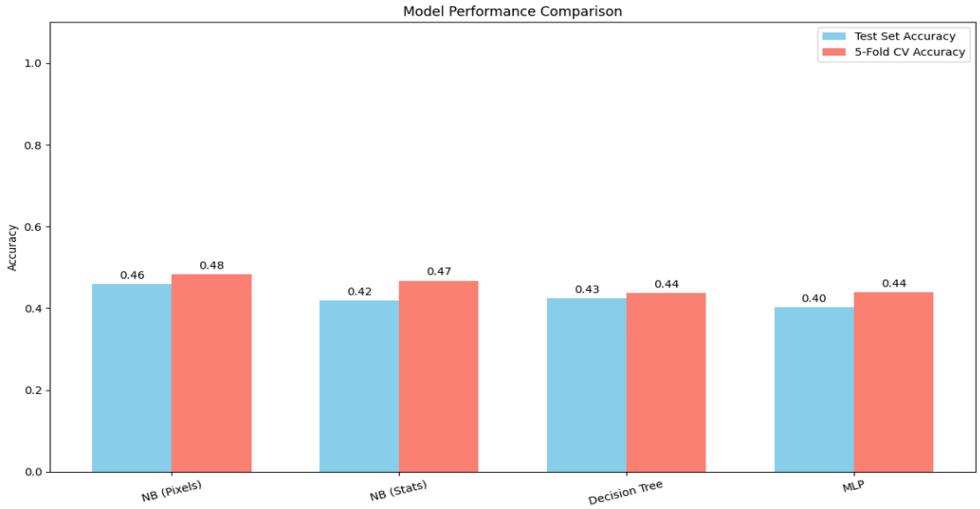
This confusion matrix displays the results of the MLP (Multi-Layer Perceptron) Classifier. The model demonstrates improved overall performance compared to Naive Bayes and Decision Tree classifiers, especially in correctly predicting the "dog" and "mouse" classes. While there is still some confusion between classes—such as "mouse" being misclassified as "cat" 20 times—the neural network shows stronger capability in capturing more complex patterns in the image data, highlighting its strength in image classification tasks.

- 5



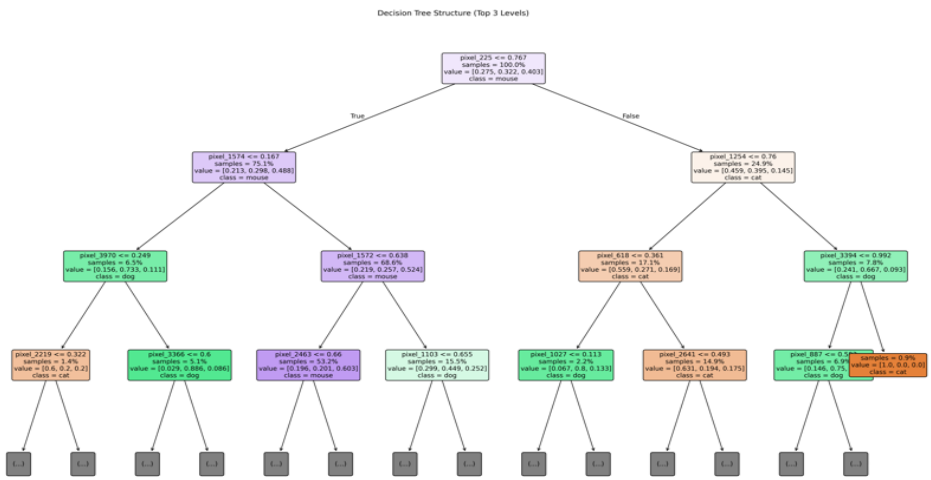
This learning curve illustrates the loss reduction of the MLP classifier over training iterations. The loss drops rapidly within the first 20 iterations and gradually approaches zero, indicating that the model is learning effectively. The smooth and consistent decline in loss suggests good convergence and stable training behavior, which is essential for achieving high classification accuracy on image data.

- 6



This bar chart compares the accuracy of four image classification models: Naive Bayes (Pixels), Naive Bayes (Stats), Decision Tree, and MLP Classifier. Each model's performance is evaluated using both test set accuracy and 5-fold cross-validation accuracy. The Naive Bayes model using pixel features achieved the highest overall accuracy (0.48 in CV), while the MLP and Decision Tree showed similar but slightly lower results. These findings suggest that simple models like Naive Bayes can be competitive on low-resolution image data, especially when tuned appropriately.

- 7



This diagram visualizes the top three levels of the trained Decision Tree used for image classification. Each node represents a decision based on a specific pixel intensity threshold (e.g., `pixel_1374 <= 0.167`). The tree splits images into different branches based on pixel values to classify them into one of the three categories: cat, dog, or mouse. While this tree structure provides interpretability and insight into which pixels influence classification decisions, it also reveals the model's reliance on low-level features, which may limit its generalization on complex or noisy images.

Conclusion

In this project, we conducted a comparative study of three machine learning algorithms — **Naive Bayes**, **Decision Tree**, and **Multi-Layer Perceptron (MLPClassifier)** — applied to the task of image classification. The dataset consisted of 866 labeled images, evenly distributed across three visually distinct classes: *cats*, *dogs*, and *mice*. Each image was resized to 64×64 pixels and flattened into a 1D vector to be used as input for the models.

Throughout the project, we followed a consistent pipeline: loading and preprocessing images, converting them into feature vectors, encoding labels, and splitting the data into training and testing sets. We applied normalization where necessary (notably for the neural network) to ensure optimal performance.

The evaluation results revealed clear differences in model behavior:

- **Naive Bayes** was the fastest and simplest model. It achieved reasonable recall on the "mouse" class (0.74) but significantly underperformed in identifying "dogs", resulting in the lowest F1-score (0.44). Its performance reflects the limitations of the conditional independence assumption when applied to image data.
- **Decision Tree** offered some interpretability and rule-based classification logic, but it struggled with overfitting and delivered the lowest accuracy (0.43). The model had difficulty distinguishing between visually similar features in grayscale image vectors.
- **MLPClassifier** outperformed both classical models, achieving the highest overall accuracy (0.53), precision, and F1-score. Despite its longer training time and the need for feature scaling, the MLP demonstrated a better ability to capture complex patterns and generalize to unseen data.

While the absolute accuracy values may seem modest, it is important to recognize that the input data consisted of low-resolution grayscale images, with no use of convolutional filters or advanced image features. Classical machine learning models often struggle in such settings compared to modern deep learning models like CNNs.